新人串讲
Paddle ir pass和pattern的匹配机制

汪博筠

2022-11-25

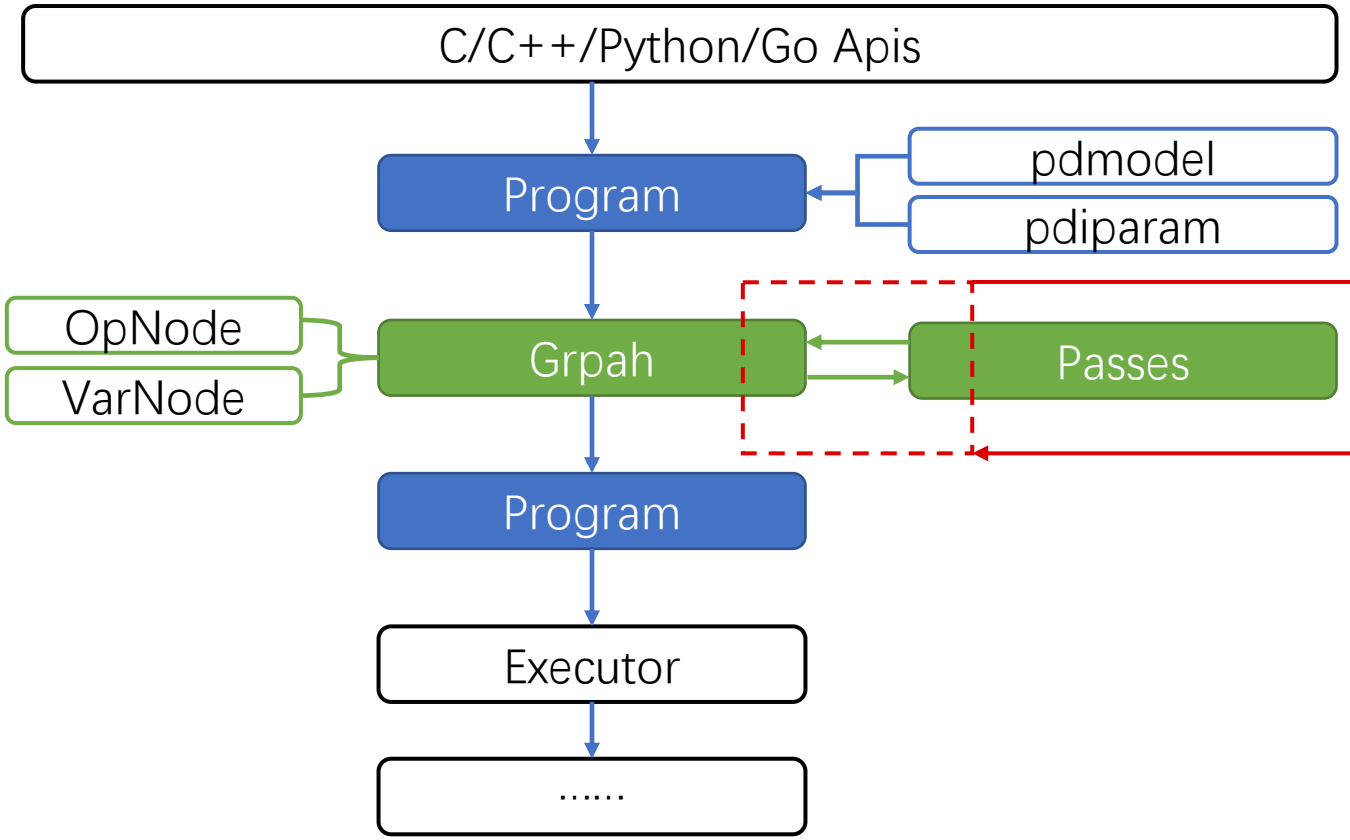
知识库:
paddle inference, ir pass 和 pattern 匹配机制初探 (baidu-int.com)

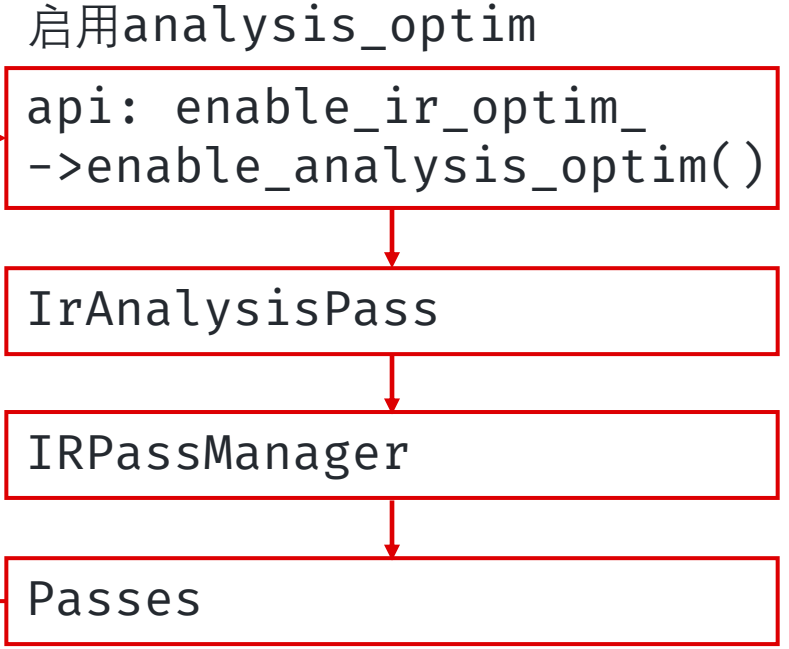
目录

- Ir pass 的执行流程
- PDPattern， PDNode， GraphPatternDetector及相关对象的构造
- GraphPatternDetector的Pattern匹配过程

1. Ir pass 的执行流程

Paddle inference [1]
paddle_inference_lib + APIs[Predictor+Config]
-> bin[-> ProgramDesc -> Program -> Graph[Passes优化]
-> Program -> executor



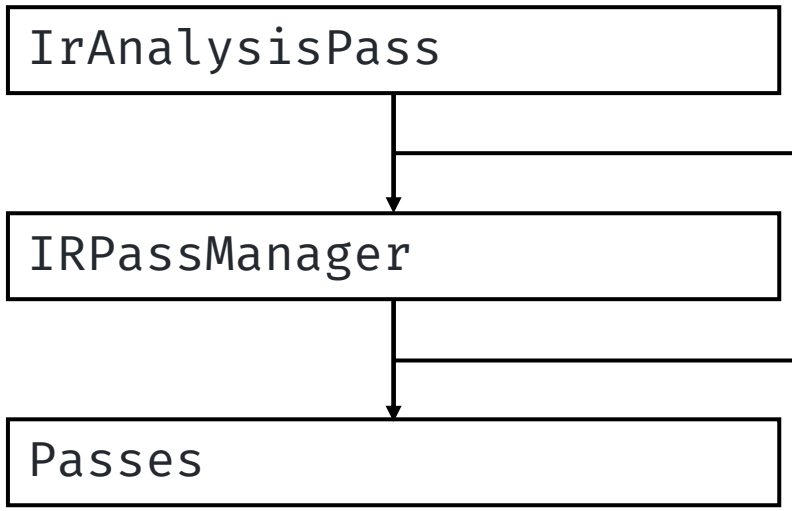


Reference:
[1] Inference Pass 现状 (baidu-int.com), 家臣

1. Ir pass 的执行流程

当确认启用analysis_optim
- 执行IrAnalysisPass
  - 对ir pass进行创建和执行
- 具体的执行链接为

IrAnalysisPass
-> IRPassManager
-> Pass

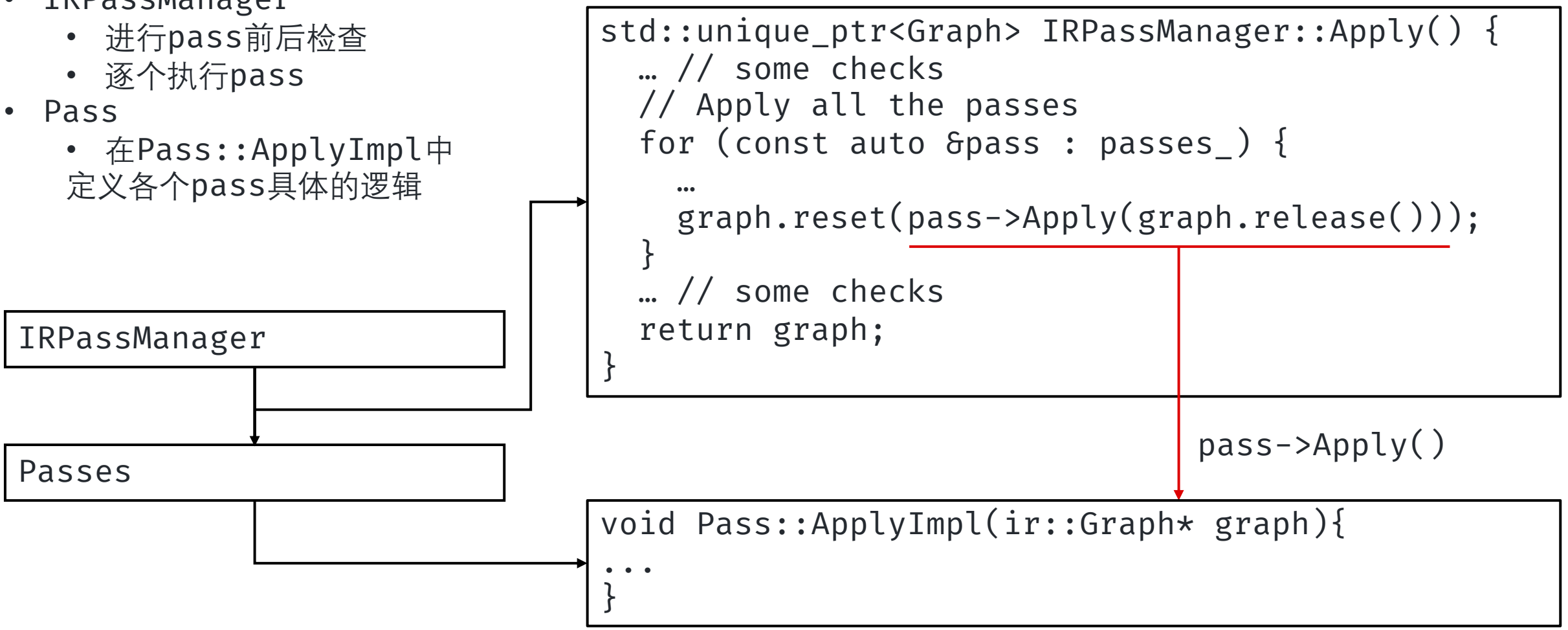



```
void IrAnalysisPass::RunImpl() {
  … // get graph
  IRPassManager the_ir_manager(argument);
  graph = the_ir_manager.Apply(std::move(graph));
  argument->SetMainGraph(graph.release());
  … // CollectFusionStatis
}
```

```
std::unique_ptr<Graph> IRPassManager::Apply() {
  … // some checks
  // Apply all the passes
  for (const auto &pass : passes_) {
    …
    graph.reset(pass->Apply(graph.release()));
  }
  … // some checks
  return graph;
}
```

1. Ir pass 的执行流程

- IRPassManager
  - 进行pass前后检查
  - 逐个执行pass
- Pass
  - 在Pass::ApplyImpl中定义各个pass具体的逻辑



1. Ir pass 的执行流程

```
void Pass::ApplyImpl(graph){
  GraphPatternDetector gpd;
  ……
  MergeLayernormPattern pattern(…);
  pattern(x);
  ……
  auto handler = [&](subg, g) {
  … // do something
  };
  ……
  gpd(graph, handler);
}
```

Ir_pass作用于 graph 的主要流程

- GraphPatternDetector
- 指定所要匹配的pattern
- 指定匹配成功后pass所需进行的变换
- 使用GraphPatternDetector执行匹配和变换

目录

- Ir pass 的执行流程
- PDPattern， PDNode， GraphPatternDetector及相关对象的构造
- GraphPatternDetector的Pattern匹配过程

2. PDPattern, PDNode, GraphPatternDetector及相关对象构造

```
class GraphPatternDetector {
pubilc:
    void operator()(Graph* graph, handle_t handler);
    private:
    ……
    PDPattern pattern_;
    std::map<PDNode, Node> pdnodes2nodes_;
}
```

GraphPatternDetector

- PDPattern类变量pattern_，由于存储GraphPatternDetector在匹配时所使用的pattern

PDPattern

- 用PDNode和PDNode之间的关联(edge)来定义相关联的图
- nodes_和edges_这两个变量保存了上述两个实体.
- 对PDPattern的操作比较简单，
- 创建新节点NewNode()和添加节点之间的关联AddEdge().

```
class PDPattern {

void AddEdge(PDNode* a, PDNode* b);
PDNode* NewNode(……)

private:
std::vector<std::unique_ptr<PDNode>
> nodes_;
std::vector<edge_t> edges_;
}
```

2. PDPattern, PDNode, GraphPatternDetector及相关对象构造

PDNode为pattern中的节点对象，
标识pattern在匹配指定ir::Node的过程中需要满足的限制条件。
PDNode包含如下属性

```
Class PDNode{
private:
  teller_t teller_;
  std::vector<teller_t> asserts_;
  PDPattern* pattern_;
  std::string name_;
  Type type_;
  Role role_{Role::kUnknown};
}
```

其中的tell_和asserts_均为PDNode的限制条件

```
teller_t = std::function<bool(Node*)>;
```

这些限制条件接受一个ir Node，返回是否能够被所属的PDNode匹配

- teller_t和asserts_在功能上有重复的地方

```
bool Tell(Node* node) const {
  if (teller_) return teller_(node);
  for (auto& asrt : asserts_) {
    if (!asrt(node)) return false;
  }
  return true;
}
```

在 PDNode::Tell 中会先判断teller_的情况，然后逐一判断asserts_的条件，并且teller_会覆盖asserts_中的设定（直接return）

2. PDPattern, PDNode, GraphPatternDetector及相关对象构造

PDNode为pattern中的节点对象，
标识pattern在匹配指定ir::Node的过程中需要满足的限制条件。
PDNode包含如下属性

```
Class PDNode{
private:
  teller_t teller_;
  std::vector<teller_t> asserts_;
  PDPattern* pattern_;
  std::string name_;
  Type type_;
  Role role_{Role::kUnknown};
}
```

```
enum class Role {
  kUnknown, // No role,
  kInput, // an input and will be retained,
  kOutput, // an output and will be retained,
  kIntermediate // will be removed after handler.
};
```

- `role_` 用于设定这个node为输入/输出还是中间值，在后续 ValidateByNodeRole 中用于识别不符合pattern定义的的subgraph.
- type_似乎暂时没有使用到。

2. PDPattern, PDNode, GraphPatternDetector及相关对象构造

graph_pattern_dector.cc中
提供了一系列helper函数辅助对PDNode的asserts_和role_进行赋予

```
PDNode* assert_is_op();
PDNode* assert_is_op(op_type);
PDNode* assert_is_var();
PDNode* assert_is_not_ctrl_var();
PDNode* assert_var_not_persistable();
PDNode* assert_is_persistable_var();
…
…
```

PDPattern中edge的设定

- 通过LinksTo/LinksFrom进行LinksTo /LinksFrom
- 通过PDPattern的AddEdge设置PDNodes之间的连接关系(DAG)

```
PDNode &PDNode::LinksTo(vector<PDNode*>) {
  for (PDNode *x : others) {
    pattern_->AddEdge(this, x);
  }
  return *this;
}
```

目录

- Ir pass 的执行流程
- PDPattern，PDNode，GraphPatternDetector及相关对象的构造
- GraphPatternDetector的Pattern匹配

3. GraphPatternDetector的Pattern匹配

使用GraphPatternDetector
对图进行的匹配和变换操作

```
void Pass::ApplyImpl(graph){
  GraphPatternDetector gpd;
  ……
  MergeLayernormPattern pattern(…);
  pattern(x);
  ……
  auto handler = [&](subg, g) {
  … // do something
  };
  ……
  gpd(graph, handler);
}
```

```
GraphPatternDetector::operator()(graph,
handler) {
  // 匹配
  if (!MarkPDNodesInGraph(*graph)) {
    return;
  }
  auto subgraphs = DetectPatterns();
  UniquePatterns(&subgraphs);
  SortSubgraphs(&subgraphs);
  RemoveOverlappedMatch(&subgraphs);
  ValidateByNodeRole(&subgraphs);
  //变换
  int id = 0;
  for (auto &g : subgraphs) {
    handler(g, graph);
  }
}
```

3. GraphPatternDetector的Pattern匹配

匹配包含以下几个过程的顺序执行

```
MarkPDNodesInGraph
DetectPatterns
UniquePatterns
SortSubgraphs
RemoveOverlappedMatch
ValidateByNodeRole
```

```
MarkPDNodesInGraph(const ir::Graph &graph) {
  for (auto &node : GraphTraits::DFS(graph)) {
    for (const auto &pdnode : pattern_.nodes()) {
      if (pdnode->Tell(&node)) {
        pdnodes2nodes_[pdnode.get()].insert(&node);
      }
    }
  }
  …… // checks for early stop
}
```

MarkPDNodesInGraph

- 使用 DFS 遍历查找符合PDPattern中每个PDNode的Tell方法(teller_和assert_)
- 将匹配上的所有IR_Node，存储在pdnodes2nodes_中
- 这个过程中未考虑PDNode和Node之间边的匹配

匹配包含以下几个过程的顺序执行

```
MarkPDNodesInGraph
DetectPatterns
UniquePatterns
SortSubgraphs
RemoveOverlappedMatch
ValidateByNodeRole
```

- 首先选择一个PDNode作为匹配的首节点
- 将首节点(PDNode)的对应的所有IRNode初始化为一个HitGroup，并将所有HitGroup记录到Init_groups中
- Hitgroup中记录了PDNode到Node(IR Node)的map，称为roles；以及命中（hit）的Node（IR Node）记录（nodes_）

```
DetectPatterns() {
  …… // some init

  auto *first_pnode = pattern_.edges().empty() ?
      pattern().nodes().front().get() :
          pattern_.edges().front().first;

  for (auto *node : pdnodes2nodes_[first_pnode]) {
    HitGroup group;
    group.roles[first_pnode] = node;
    init_groups.emplace_back(group);
  }
  ….
}
```

3. GraphPatternDetector的Pattern匹配

匹配包含以下几个过程的顺序执行

```
MarkPDNodesInGraph
DetectPatterns
UniquePatterns
SortSubgraphs
RemoveOverlappedMatch
ValidateByNodeRole
```

- bi_records
- 用于进行hitgroups关于edge的递推
分为pre_groups(groups[i-1]) 和
cur_groups(groups[i])
Groups[i] = groups[i-1] +
matchedEdges.nodes
- 枚举 pattern 中涉及的边两端的Node中所有相连的IR Node
- 与 group 匹配成功的相连的IR Node将被添加进本次递推的结果中

```
DetectPatterns() {
  …… // some init

  for (const auto &edge : pattern_.edges()) {
    auto &pre_groups = bi_records[step % 2];
    auto &cur_groups = bi_records[1 - (step++ % 2)];
    cur_groups.clear();
    if (pre_groups.empty()) break;
    for (Node *source : pdnodes2nodes_[edge.first]) {
      for (Node *target : pdnodes2nodes_[edge.second]) {
        for (const auto &group : pre_groups) {
          if (IsNodesLink(source, target)) {
            HitGroup new_group = group;
            bool flag = new_group.Match(source, edge.first) &&
              new_group.Match(target, edge.second);
            if (flag) {
              new_group.Register(source, edge.first);
              new_group.Register(target, edge.second);
              cur_groups.push_back(new_group);
            }
          }
        }
      }
    }
  }
  ……
}
```

3. GraphPatternDetector的Pattern匹配

匹配包含以下几个过程的顺序执行

```
MarkPDNodesInGraph
DetectPatterns
UniquePatterns
SortSubgraphs
RemoveOverlappedMatch
ValidateByNodeRole
```

```
bool Match(Node *node, PDNode *pat) {
  if (nodes_.count(node)) {
  if (roles.count(pat) && roles[pat] == node) return true;
    return false;
  } else {
  if (roles.count(pat) && roles[pat] != node) return false;
    return true;
  }
}

bool flag = new_group.Match(source, edge.first) &&
            new_group.Match(target, edge.second);
```

Hitgroup match:

```
Node in group, PDNode in group, PDNode map to Node          -> true
Node in group, PDNode not in group                          -> false
Node in group, PDNode not map to Node                       -> false
---
Node not in group, PDNode not in group                      -> true
Node not in group, PDNode not map to Node                   -> true
Node not in group, PDNode in group, PDNode not map to Node -> false
```

3. GraphPatternDetector的Pattern匹配

```
MarkPDNodesInGraph
DetectPatterns
UniquePatterns
SortSubgraphs
RemoveOverlappedMatch
ValidateByNodeRole
```

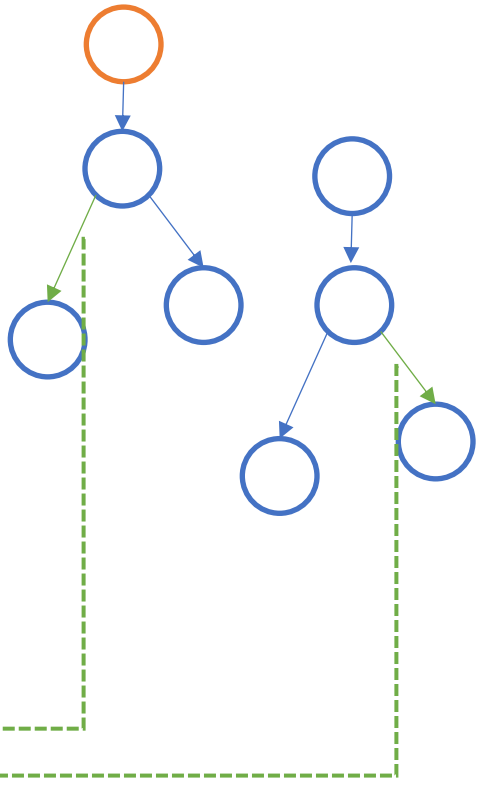
```
bool flag = new_group.Match(source, edge.first) &&
            new_group.Match(target, edge.second);
```

```
Flags = true <-
  [one       ] Node in group, PDNode in group, PDNode map to Node
  [the ohter] Node in group, PDNode in group, PDNode map to Node
  (impossible in ir dag? How about ring?)

  [one       ] Node in group, PDNode in group, PDNode map to Node
  [the ohter] Node not in group, PDNode not in group

  [one       ] Node in group, PDNode in group, PDNode map to Node
  [the ohter] Node not in group, PDNode not map to Node

  [one       ] Node not in group, PDNode not in group
  [the ohter] Node not in group, PDNode not map to Node
```



3. GraphPatternDetector的Pattern匹配

匹配包含以下几个过程的顺序执行

```
MarkPDNodesInGraph
DetectPatterns
UniquePatterns
SortSubgraphs
RemoveOverlappedMatch
ValidateByNodeRole
```

DetectPatterns

- 可能会匹配到一些不正确的subgraph

丢弃非法subgraph

- UniquePatterns
- RemoveOverlappedMatch
- ValidateByNodeRole

UniquePatterns通过排序和hash的方式识别重复的subgraphs

```
UniquePatterns(subgraphs) {
  for (auto &g : *subgraphs) {
    std::sort(sorted_keys.begin(),
              sorted_keys.end(),
              GraphItemLessThan());
    std::stringstream ss;
      for (auto &item : sorted_keys) {
        ss << item.first << ":" << item.second;
      }
    auto key = hasher(ss.str());
    if (!set.count(key)) {
      result.emplace_back(g);
    set.insert(key);
    }
  }
  *subgraphs = result;
}
```

3. GraphPatternDetector的Pattern匹配

匹配包含以下几个过程的顺序执行

```
MarkPDNodesInGraph
DetectPatterns
UniquePatterns
SortSubgraphs
RemoveOverlappedMatch
ValidateByNodeRole
```

DetectPatterns
- 可能会匹配到一些不正确的subgraph

丢弃非法subgraph
- UniquePatterns
- RemoveOverlappedMatch
- ValidateByNodeRole

RemoveOverlappedMatch用于去除交叠的subgraphs
- 首先使用SortSubgraphs将subgraphs排序
- 排序后遍历subgraphs
- 通过记录已匹配的IRNode去除重复匹配IRNode的subgraph

```
RemoveOverlappedMatch(*subgraphs) {
  for (const auto &subgraph : *subgraphs) {
    bool valid = true;
    for (auto &item : subgraph) {
      if (item.first->IsIntermediate() &&
            node_set.count(item.second)) {
        valid = false;
        break;
      }
    }
    if (valid) {
      for (auto &item : subgraph) {
        node_set.insert(item.second);
      }
      result.push_back(subgraph);
    }
  }
  *subgraphs = result;
}
```

3. GraphPatternDetector的Pattern匹配

匹配包含以下几个过程的顺序执行

```
MarkPDNodesInGraph
DetectPatterns
UniquePatterns
SortSubgraphs
RemoveOverlappedMatch
ValidateByNodeRole
```

DetectPatterns

- 可能会匹配到一些不正确的subgraph

丢弃非法subgraph

- UniquePatterns
- RemoveOverlappedMatch
- ValidateByNodeRole

ValidateByNodeRole检查subgraph中Role为Intermediate的PDNode所匹配到的IRNode是否连接到了subgraph内的非Intermediate的节点(包含前后连接)

```
ValidateByNodeRole(subgraphs) {
subgraphs->erase(std::remove_if(…,
    [](&subgraph) -> bool {
      std::set<Node *> ios;
      for (auto &item : subgraph) {
        if (!item.first->IsIntermediate()) {
          ios.insert(item.second);
        }}
      for (auto &item : subgraph) {
        if (item.first->IsIntermediate()) {
          for (auto *x : item.second->inputs) {
            if (!ios.count(x)) {
              return true;
            }}
          for (auto *x : item.second->outputs) {
            if (!ios.count(x)) {
              return true;
            }}}}
    return false;}),
    subgraphs->end());
}
```

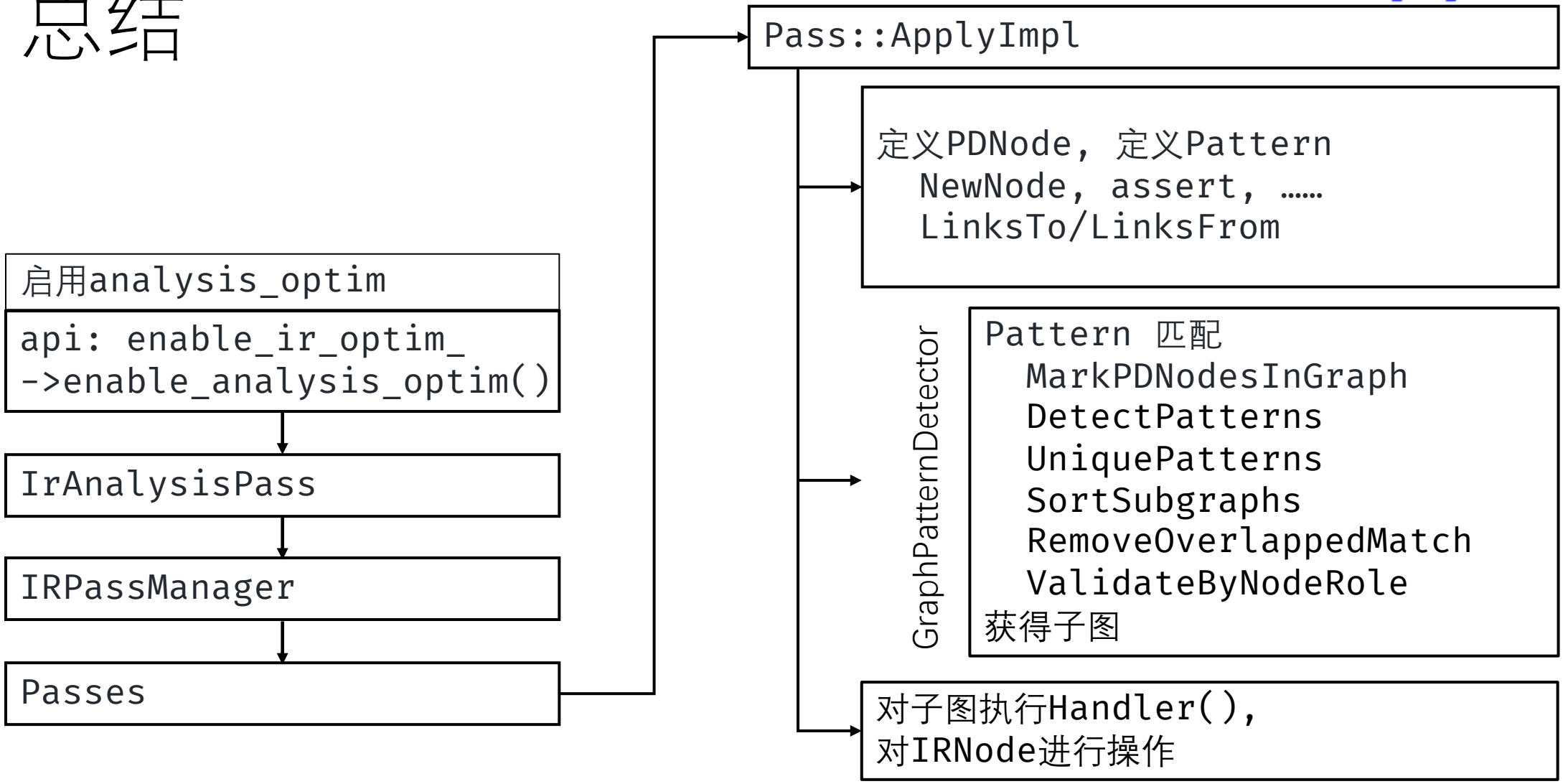
3. GraphPatternDetector的Pattern匹配

使用GraphPatternDetector
对图进行的匹配和变换操作

```
void Pass::ApplyImpl(graph){
  GraphPatternDetector gpd;
  ……
  MergeLayernormPattern pattern(…);
  pattern(x);
  ……
  auto handler = [&](subg, g) {
  … // do something
  };
  ……
  gpd(graph, handler);
}
```

```
GraphPatternDetector::operator()(graph,
handler) {
  // 匹配
  if (!MarkPDNodesInGraph(*graph)) {
    return;
  }
  auto subgraphs = DetectPatterns();
  UniquePatterns(&subgraphs);
  SortSubgraphs(&subgraphs);
  RemoveOverlappedMatch(&subgraphs);
  ValidateByNodeRole(&subgraphs);
  //变换
  int id = 0;
  for (auto &g : subgraphs) {
    handler(g, graph);
  }
}
```

总结


启用analysis_optim
api: enable_ir_optim_
->enable_analysis_optim()
IrAnalysisPass
IRPassManager
Passes
Pass::ApplyImpl
定义PDNode, 定义Pattern
NewNode, assert, ……
LinksTo/LinksFrom
GraphPatternDetector
Pattern 匹配
MarkPDNodesInGraph
DetectPatterns
UniquePatterns
SortSubgraphs
RemoveOverlappedMatch
ValidateByNodeRole
获得子图
对子图执行Handler(),
对IRNode进行操作

Thanks & QA